

Day 31 — Student Performance & Learning Analytics

PDF generated directly from the Day 31 Jupyter Notebook (.ipynb)

Day 31 — Student Performance & Learning Analytics

Veda Technology · Level 2 · Day 31 · Project 1 of 3

Analyze student academic and learning data to identify performance patterns, subject strengths and weaknesses, attendance trends, and factors associated with successful outcomes.

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, Jupyter Notebook.

1. Import Libraries

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, accuracy_score, classification_report, confusion_matrix
sns.set_theme(style="whitegrid")
```

2. Create Synthetic Student Dataset

Code

```
np.random.seed(42)
n=200
df=pd.DataFrame({
    "student_id":range(1,n+1),
    "attendance":np.clip(np.random.normal(78,10,n),45,100),
    "study_hours":np.clip(np.random.normal(3.5,1.5,n),0.5,8),
    "assignment_score":np.clip(np.random.normal(72,12,n),30,100),
    "previous_score":np.clip(np.random.normal(68,13,n),25,100),
    "math_score":np.clip(np.random.normal(70,14,n),20,100),
    "science_score":np.clip(np.random.normal(72,13,n),20,100),
    "english_score":np.clip(np.random.normal(74,12,n),20,100)
})
df["final_exam_score"]=np.clip(0.25*df.attendance+2.2*df.study_hours+0.18*df.assignment_score+0.22*df.previous_score+0.12*df.math_score+0.1*df.science_score+0.08*df.english_score,0,100)
df["passed"]=(df.final_exam_score>=50).astype(int)
df.head()
```

3. Inspection and Cleaning

Code

```
print("Shape:",df.shape)
print(df.isnull().sum())
print(df.describe().round(2))
df=df.drop_duplicates().copy()
num=df.select_dtypes(include=np.number).columns
df[num]=df[num].fillna(df[num].median())
```

4. Exploratory Data Analysis

Code

```
plt.figure(figsize=(8,5))
sns.histplot(df.final_exam_score,bins=20,kde=True)
plt.title("Final Exam Score Distribution"); plt.show()

plt.figure(figsize=(8,5))
sns.scatterplot(data=df,x="attendance",y="final_exam_score")
plt.title("Attendance vs Final Exam Score"); plt.show()

plt.figure(figsize=(8,5))
sns.scatterplot(data=df,x="study_hours",y="final_exam_score")
plt.title("Study Hours vs Final Exam Score"); plt.show()
```

5. Subject-Wise Performance

Code

```
subjects=["math_score","science_score","english_score"]
```

```
means=df[subjects].mean().sort_values(ascending=False)
print(means.round(2))
means.plot(kind="bar",figsize=(8,5),title="Average Subject Scores")
plt.ylabel("Average Score"); plt.xticks(rotation=0); plt.show()
```

6. Attendance Analysis

Code

```
groups=pd.cut(df.attendance,[0,60,75,90,100],labels=["Below 60%","60-75%","75-90%","90-100%"])
attendance_summary=df.groupby(groups,observed=False).final_exam_score.agg(["count","mean"]).round(2)
attendance_summary
```

7. Correlation Analysis

Code

```
corr=df.drop(columns="student_id").corr(numeric_only=True)
plt.figure(figsize=(10,7))
sns.heatmap(corr,annot=True,fmt=".2f",cmap="coolwarm")
plt.title("Correlation Matrix"); plt.show()
display(corr.final_exam_score.sort_values(ascending=False).to_frame("correlation"))
```

8. Feature Engineering

Code

```
df["average_subject_score"]=df[subjects].mean(axis=1)
df["attendance_study_index"]=df.attendance*df.study_hours
df["assignment_previous_avg"]=df[["assignment_score","previous_score"]].mean(axis=1)
df.head()
```

9. Linear Regression — Final Score Prediction

Code

```
features=["attendance","study_hours","assignment_score","previous_score","math_score","science_score","english_score","average_subject_score"]
X=df[features]; y=df.final_exam_score
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.20,random_state=42)
model=LinearRegression().fit(Xtr,ytr)
pred=model.predict(Xte)
print("MAE:",round(mean_absolute_error(yte,pred),2))
print("RMSE:",round(np.sqrt(mean_squared_error(yte,pred)),2))
print("R²:",round(r2_score(yte,pred),3))
```

10. Logistic Regression — Pass/Fail Prediction

Code

```
X=df[features]; y=df.passed
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.20,random_state=42,stratify=y)
log=LogisticRegression(max_iter=2000).fit(Xtr,ytr)
p=log.predict(Xte)
print("Accuracy:",round(accuracy_score(yte,p),3))
print(classification_report(yte,p))
print("Confusion Matrix:\n",confusion_matrix(yte,p))
```

11. Feature Interpretation

Code

```
coef=pd.Series(model.coef_,index=features).sort_values(key=abs,ascending=False)
display(coef.to_frame("coefficient"))
coef.sort_values().plot(kind="barh",figsize=(9,5),title="Linear Regression Feature Coefficients")
plt.show()
```

12. Basic Student Risk Analysis

Students are classified into High, Medium, or Low risk using simple attendance and final-score thresholds.

Note: This is a learning demonstration using synthetic data and must not be used for real student decisions.

Code

```
risk=df[["student_id","attendance","study_hours","average_subject_score","final_exam_score","passed"]].copy()
risk["risk_level"]=np.select([
    (risk.attendance<60)|(risk.final_exam_score<40),
    (risk.attendance<75)|(risk.final_exam_score<50)],
    ["High","Medium"],default="Low")
display(risk.risk_level.value_counts().to_frame("students"))
display(risk.sort_values("final_exam_score").head(10))
```

13. Interview Questions

1. What is EDA?
2. How do you handle missing student records?
3. What is correlation? Does it imply causation?
4. When would you use linear regression?
5. When would you use logistic regression?
6. Why separate training and testing data?
7. What do MAE, RMSE, and R^2 measure?
8. What is feature engineering?
9. How can attendance be analyzed against performance?
10. How would you identify students needing academic support?

Conclusion

This project demonstrates an end-to-end introductory data science workflow: cleaning, EDA, visualization, attendance and subject analysis, correlation, feature engineering, regression/classification, evaluation, and basic risk analysis.